

System Installation und Störungsbehebungen

Grundlagen Bicep & Repetition SIUS



September 2026 / Markus Nussbaumer

Agenda

Themen

1. Repetition IaC – 15 Min.

Imperativ/deklarativ, IaC, Vorteile

2. IaC & Bicep – 15 Min.

Grundidee und Einsatz von Bicep

3. Bicep Crashkurs – Video – 25 Min.

4. Bicep-Template verstehen – 15 Min.

5. Selbststudium: Bicep ausprobieren – 40 Min.

6. Repetition / Vorbereitung Modulprüfung – 90 Min.

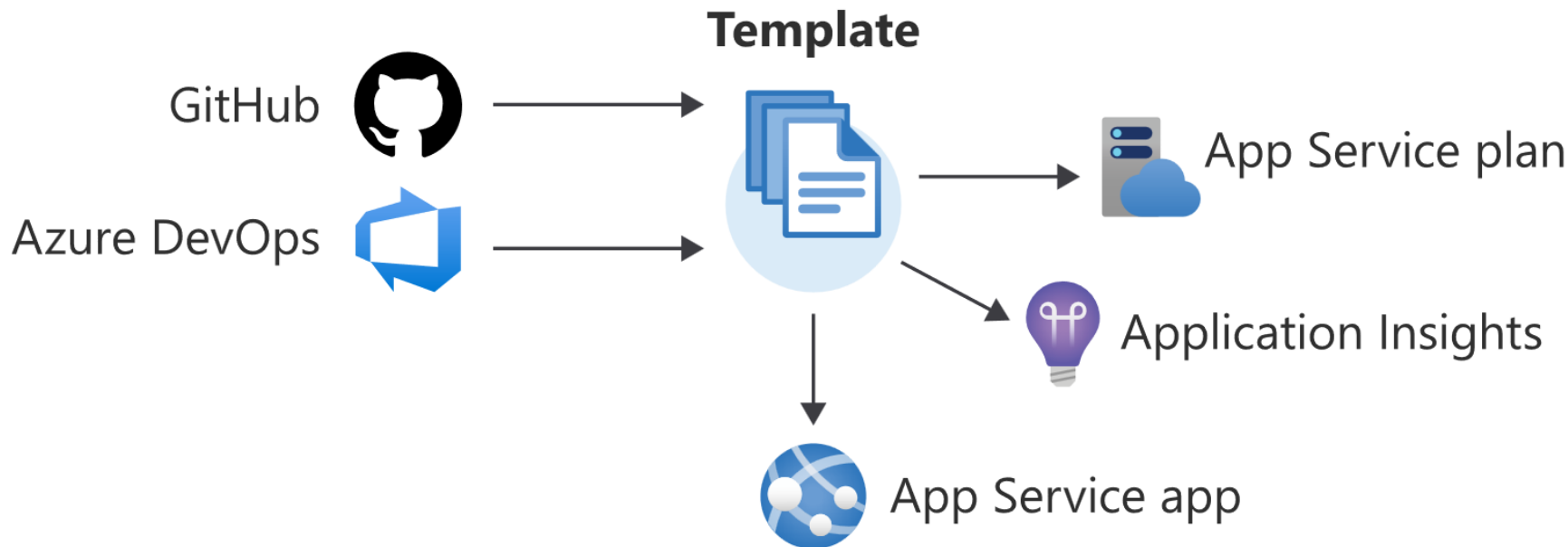
5 Gruppen, kurze Vorbereitung und Präsentation/Aktivierung

7. Abschluss

Agenda

Repetition

- Was ist ein Sprach Modell?
- Was ist ein Encoder in einem Sprachmodell?
- Was ist ein Decoder in einem Sprachmodell?
- Nennen Sie zwei Beispiele für eine Generative KI.



Infrastructure as Code

Was bedeutet das?

Infrastructure as Code (IaC) bedeutet:

- Infrastruktur wird als Code beschrieben statt manuell über eine Benutzeroberfläche eingerichtet.

Beispiele für Infrastruktur:

- Virtuelle Maschinen
- Netzwerke
- Storage Accounts
- Datenbanken
- Berechtigungen

Grundidee:

- Code → Azure → Infrastruktur

Konzept: *Wir beschreiben den gewünschten Zustand – Azure setzt ihn um.*

Manuell vs. Infrastructure as Code

Manuell im Azure Portal

Azure Portal → Ressource auswählen → Einstellungen vornehmen → Erstellen

Mit Infrastructure as Code

Bicep-Datei → Deployment starten → Azure erstellt die Ressourcen

Vorteile von IaC:



Wiederholbar



Automatisierbar



Dokumentiert



Änderungen nachvollziehbar



Im Team nutzbar



Weniger manuelle Fehler

Was ist Bicep?

- Bicep ist eine Sprache für **Infrastructure as Code in Azure**
- Wir beschreiben, **welche Ressourcen** benötigt werden
- Azure übernimmt die Bereitstellung
- Ein Bicep-Template ist vergleichbar mit einem **Bauplan**

Was steht in einem Bicep-Template?

Ein Bicep-Template beschreibt beispielsweise:

Wo?

Region / Standort

Was?

Speicher, Netzwerk, Server usw.

Wie?

Grösse und gewünschte Eigenschaften

Welche Bezeichnung?

Namen der Ressourcen

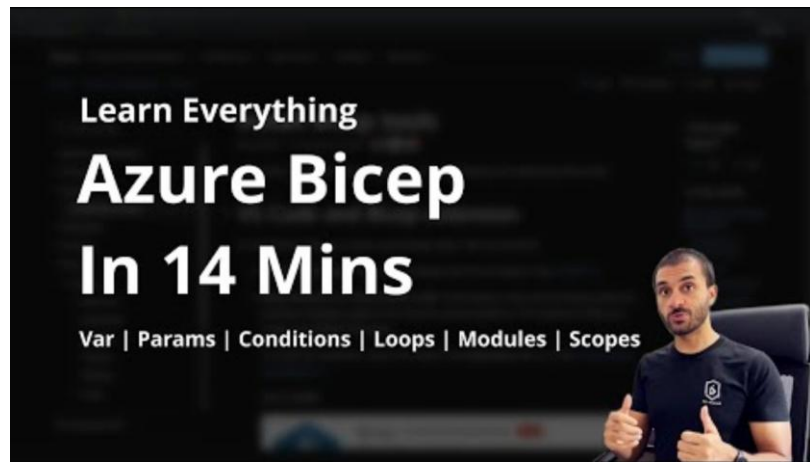
 Das Template ist damit vergleichbar mit einem **Bauplan für die Cloud-Infrastruktur.**

Azure Bicep Crashkurs – Schritt für Schritt

Achtet beim Video besonders auf:

- Was ist IaC?
- Wie funktioniert Bicep grundsätzlich?
- Wie ist ein Bicep-Template aufgebaut?
- Was sind Parameter und Ressourcen?
- Wozu dienen Module?

 **Notiert mindestens 3 wichtige Erkenntnisse.**



<https://www.youtube.com/watch?v=mKG5d9rnaYg>

- **bicep = Infrastructure as Code für Azure**
- Infrastruktur wird **als Code beschrieben**, statt alles manuell im Azure Portal anzulegen.
- Bicep arbeitet **deklarativ**: Man beschreibt den gewünschten Zustand, nicht jeden einzelnen Arbeitsschritt.
- Eine **.bicep-Datei** enthält die Definition der Azure-Ressourcen.
- Wichtigste Elemente erkennen: **resource, name, location, properties** sowie **Parameter und Variablen**.
- Ressourcen können **voneinander abhängig** sein und gemeinsam bereitgestellt werden.
- **Module** helfen, grössere Templates aufzuteilen und Teile wiederzuverwenden.
- Das Template kann erneut ausgeführt werden – dadurch wird Infrastruktur **reproduzierbar und standardisiert**.
- Bicep wird über Azure bereitgestellt; im Hintergrund basiert die Bereitstellung auf **Azure Resource Manager (ARM)**.

Warum Module?

Bei grösseren Infrastrukturen wird ein Bauplan schnell umfangreich.

Deshalb kann Bicep in **Module** aufgeteilt werden:

Gesamtlösung

- Netzwerk
- Speicher
- Server
- Datenbank

Vorteil:

Die einzelnen Bausteine bleiben übersichtlich und können wiederverwendet werden.

Ein Modul ist ein wiederverwendbarer Baustein der Infrastruktur.

Gute Praxis mit Bicep

Ein gutes Bicep-Projekt sollte:

- verständlich aufgebaut sein
- wiederverwendbar sein
- Änderungen nachvollziehbar machen
- grosse Lösungen in kleinere Bausteine aufteilen
- keine Passwörter direkt im Code speichern
- für verschiedene Umgebungen einsetzbar sein

Ziel:

- Infrastruktur soll **einfach, reproduzierbar und nachvollziehbar** bereitgestellt werden.

Aufbau eines Bicep-Templates

Ein Bicep-Template besteht typischerweise aus:

Parameter: Werte, die beim Deployment verändert werden können.

```
param location string = 'switzerlandnorth'
```

Variablen: Interne Werte für das Template.

```
var storageName = 'hfinfdemo12345'
```

Ressourcen: Die Infrastruktur, die erstellt werden soll.

```
resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
```

```
  ...
```

```
}
```

Outputs

Werte, die nach dem Deployment ausgegeben werden.

```
output storageId string = storage.id
```

Vom Code zur Azure-Ressource

Der Ablauf eines Deployments:

1. Bicep-Datei erstellen

main.bicep



2. Infrastruktur beschreiben

Storage, Netzwerk, VM usw.



3. Bicep wird verarbeitet



4. Azure Resource Manager (ARM)



5. Ressourcen werden erstellt oder angepasst

Wichtig:

Bicep ist eng mit dem **Azure Resource Manager** verbunden.

Die wichtigsten Modulinhalte nochmals aktivieren, Zusammenhänge erkennen und Wissenslücken schliessen. Ziel ist eine gezielte Vorbereitung auf die Modulprüfung.

Organisatorische Hinweise

- 5 Gruppenräume – jede Gruppe bearbeitet einen Themenbereich
- 45 Minuten Vorbereitungszeit
- Inhalte auf das Wesentliche reduzieren
- Fokus auf Verstehen, Erklären und Anwenden
- Praxisbeispiele einbauen
- Klasse kurz aktivieren und einbeziehen
- 8 Minuten Präsentationszeit pro Gruppe

Zeitbedarf: 45 Min. Vorbereitung + 40 Min. Präsentationen = **85 Minuten**

Selbststudium: Bicep praktisch ausprobieren

Vom Konzept zur ersten Bicep-Datei

Ihr habt **Infrastructure as Code (IaC)** und **Bicep** kennengelernt. Jetzt probiert ihr es selbst aus.

Euer Auftrag:

-  Bicep in **Visual Studio Code** vorbereiten
-  eine erste **Bicep-Datei** erstellen
-  einen **Azure Storage Account** als Code beschreiben
-  Aufbau und Syntax nachvollziehen
-  kleine Änderungen und Fehler ausprobieren

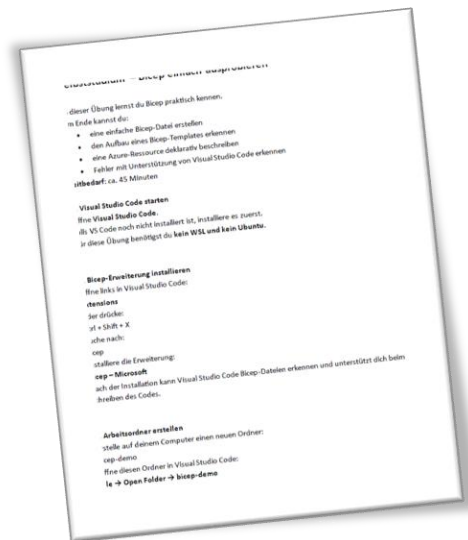
Wichtig:

Arbeitet Schritt für Schritt nach der Anleitung.

Für diese Übung braucht ihr kein WSL / Ubuntu.

Es geht noch nicht um ein echtes Deployment, sondern darum, Bicep praktisch kennenzulernen.

 **Zeit: ca. 30 Minuten**



Repetition & Prüfungsvorbereitung

Schwerpunkte für die Modulprüfung

- Begriffe und Zusammenhänge verständlich erklären
- Technologien und Konzepte unterscheiden und einordnen
- Vor- und Nachteile begründen
- Vorgehen anhand eines Praxisbeispiels erklären
- Lösungen bzw. Situationen analysieren und beurteilen
- Nicht nur „Was ist das?“, sondern auch „Warum und wann wird es eingesetzt?“

Repetition in 5 Expertengruppen

ipso!

Jede Gruppe repetiert ihren Themenbereich und gestaltet daraus eine 8-minütige aktive Wiederholung mit der Klasse. Die Themen orientieren sich an den Bereichen der Modulprüfung.

- **Raum 1 – Windows, Server & PowerShell**

Windows 10/11, Windows Server, Server Core, Migration, PowerShell-Grundlagen und Cmdlets.

- **Raum 2 – Active Directory, DNS & DHCP**

AD-Rollen, Gruppen, Trust, AGDLP, DNS-Abfragen/Records/TTL sowie DHCP-Prozess, Relay und Failover.

- **Raum 3 – Azure, Verfügbarkeit & Business Continuity**

Cloud vs. On-Premise, Cloud Service Types, Scaling, SLA, Entra ID, Metrics/Logs, RPO und RTO.

- **Raum 4 – Security & Identity**

TPM/BitLocker, MFA, Conditional Access, Least Privilege, RBAC, PowerShell Security, Security Baselines, LAPS, Defender und Firewall.

- **Raum 5 – Moderne Infrastruktur & Automatisierung**

Hyper-V, Container, Intune, Azure Arc, GitHub, Generative KI/LLM sowie Bicep & Infrastructure as Code.

5 Gruppen – jede Gruppe bearbeitet einen Themenbereich

45 Minuten Vorbereitungszeit

Inhalte auf das Wesentliche reduzieren

Fokus auf Verstehen, Erklären und Anwenden

Praxisbeispiele einbauen

Klasse kurz aktivieren und einbeziehen

8 Minuten Präsentationszeit pro Gruppe

Alle Gruppenmitglieder beteiligen sich

Zeitbedarf: 45 Min. Vorbereitung + 40 Min. Präsentationen = 85 Minuten

1. Repetition – ca. 5 Min.

Erklärt die wichtigsten Begriffe, Zusammenhänge und Technologien verständlich und kompakt.

2. Aktivierung – ca. 2 Min.

Bezieht die Klasse aktiv ein, z. B. mit Zuordnen, Wahr/Falsch, Begriff erklären lassen, Mini-Szenario, Abstimmung oder Fehler finden.

3. Take-away – ca. 1 Min.

Nennt zum Abschluss die 3 wichtigsten Punkte, die von eurem Themenbereich hängen bleiben sollen.

Keine reine Präsentation: Die Klasse soll bei jeder Gruppe aktiv mitdenken und mitmachen.

Auftrag: „Unsere Firma braucht eine neue Web-App in Azure. Zusätzlich soll überwacht werden, ob die Anwendung funktioniert.“

Ohne Bicep müsste ein Informatiker im Azure-Portal die Ressourcen **manuell anlegen und konfigurieren**. Mit Bicep könnte ein vereinfachter Ausschnitt so aussehen:

```
resource appPlan 'Microsoft.Web/serverfarms@2023-12-01' = {  
  name: 'mein-app-plan'  
  location: 'switzerlandnorth'  
  sku: {  
    name: 'B1'  
  }  
}
```

```
resource webApp 'Microsoft.Web/sites@2023-12-01' = {  
  name: 'meine-webapp'  
  location: 'switzerlandnorth'  
  properties: {  
    serverFarmId: appPlan.id  
  }  
}
```

Grundlagen Bicep

Vorteile und Nachteile von Infrastructure as Code

- Was sind die Vorteile von Infrastructure as Code?
- Was sind die Nachteile von Infrastructure as Code

Grundlagen Bicep

Imperative Code vs. Declarative Code

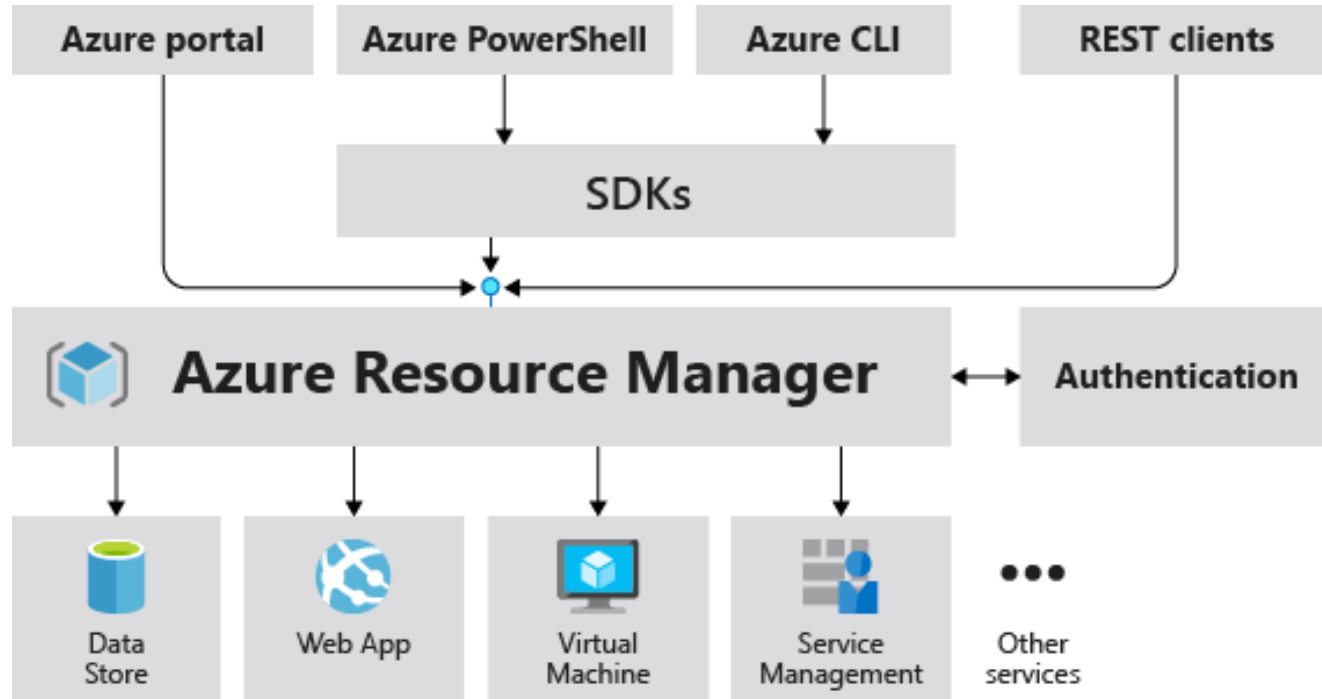
- Was ist Imperativer Code?
- Wo wird Imperativer Code verwendet?
- Was ist Deklarativer Code?
- Wo wird Deklarativer Code verwendet?

Für die Studierenden würde ich es **einfach und praxisnah** so formulieren:

	Imperativer Code	Deklarativer Code
Was ist das?	Beschreibt Schritt für Schritt, WIE etwas gemacht wird.	Beschreibt WAS am Ende vorhanden sein soll.
Beispiel	„Erstelle Ordner → installiere Software → ändere Konfiguration.“	„Ich möchte einen Webserver mit diesen Eigenschaften.“
Wo verwendet?	Python, PowerShell, Bash, Java, C#	Bicep, Terraform, ARM Templates, Kubernetes YAML, SQL
Typischer Einsatz	Skripte, Programme, Automatisierung von Abläufen	Infrastructure as Code (IaC), Cloud-Infrastruktur, Konfigurationen

Grundlagen Bicep

Azure Resource Manager



Grundlagen Bicep

Bicep



Demonstration Installation Bicep

Grundlagen Bicep

Recherche Bicep

- Was ist ARM in Kontext von Azure?
- Was sind ARM Templates und in welcher Sprache werde diese geschrieben?
- Was sind Resource Providers in Azure?
- Was ist die eindeutige Identifikation einer Azure Resource?
- Aus welchen Teilen besteht die eindeutige Identifikation einer Azure Resource?
- Was sind Child Resources in Azure?

Demonstration Bicep Resource erstellen

1. Erstellt einen Account unter learn.microsoft.com (mit eurem IPSO Account)
2. Erstellt einen Storage Account gemäss folgender Aufgabe:
<https://learn.microsoft.com/en-us/training/modules/build-first-bicep-template/4-exercise-define-resources-bicep-template?pivots=powershell>
3. Ihr könnt falls vorhanden auch eure eigene Azure Umgebung verwenden.

Demonstration Bicep Variablen und Parameter

1. Erweitert euer Bicep Template gemäss folgender Aufgabe:
<https://learn.microsoft.com/en-us/training/modules/build-first-bicep-template/6-exercise-add-parameters-variables-bicep-template?pivots=powershell>
2. Ihr könnt falls vorhanden auch eure eigene Azure Umgebung verwenden.

Demonstration Bicep Modules

1. Erweitert euer Bicep Template gemäss folgender Aufgabe:
<https://learn.microsoft.com/en-us/training/modules/build-first-bicep-template/8-exercise-refactor-template-modules?pivots=powershell>
2. Ihr könnt falls vorhanden auch eure eigene Azure Umgebung verwenden.

Demonstration Azure Verified Modules

Zusammenfassung

Reflexion

- Was habe ich heute gelernt?
- Was war das Highlight vom heutigen Tag?
- Was war gut am Unterricht?
- Was können wir am Unterricht verbessern?
 - Feedback auf per Mail möglich!
- Weitere Fragen?

Nachbereitung

- <https://learn.microsoft.com/en-us/training/modules/introduction-to-infrastructure-as-code-using-bicep/>
- <https://learn.microsoft.com/en-us/training/modules/build-first-bicep-template/>

Herzlichen Dank

